University of Cape Town



Machine Learning Assignment 1

Pamela Afful

Student Number: AFFPAM001

October 20, 2026

Department of Statistical Sciences Plagiarism Declaration form

A copy of this form, completed and signed, to be attached to all coursework submissions to the Statistical Sciences Department. Submissions without this form will not be marked.

COURSE CODE: **STA** 5068Z

COURSE NAME: MACHINE LEARNING

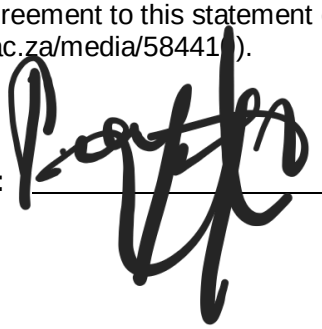
STUDENT NAME: PAMELA AFFUL

STUDENT NUMBER: AFFPAM001

PLAGIARISM DECLARATION

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used a generally accepted citation and referencing style. Each contribution to, and quotation in, this tutorial/report/project from the work(s) of other people has been attributed, and has been cited and referenced.
3. This tutorial/report/project is my own work.
4. I have not allowed, and will not allow, anyone to copy my work with the intention of passing it off as his or her own work.
5. I acknowledge that copying someone else's assignment or essay, or part of it, is wrong, and declare that this is my own work.

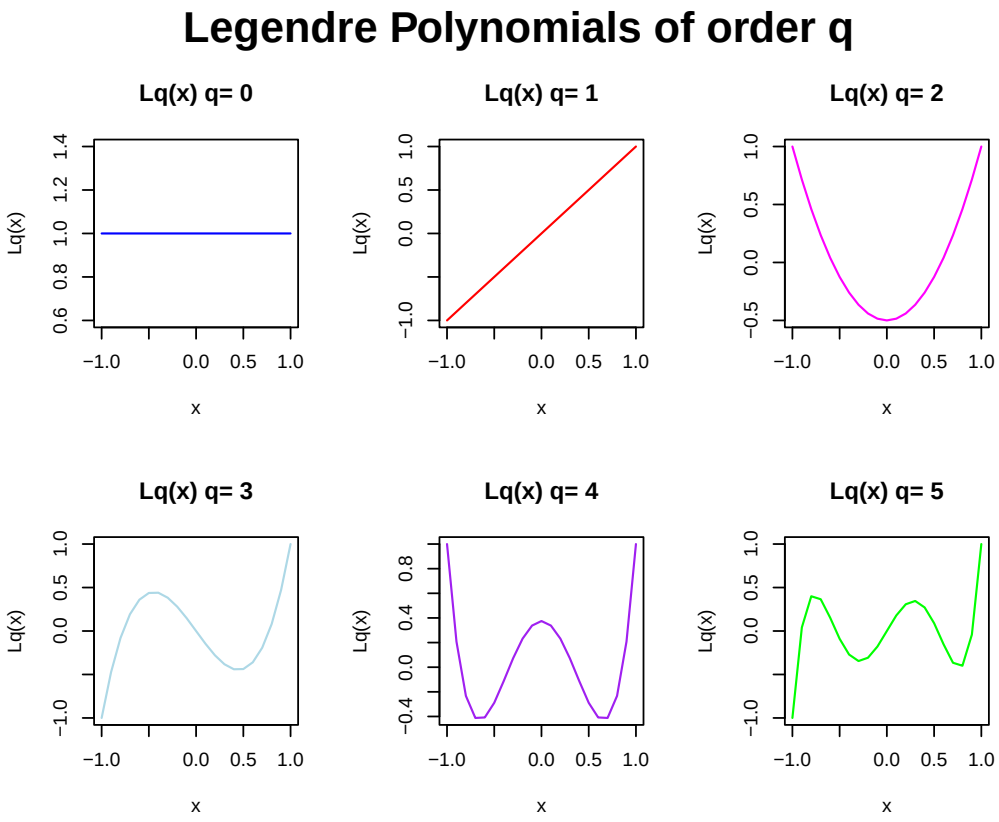
Note that agreement to this statement does not exonerate you from UCT's Academic Misconduct policy (https://uct.ac.za/media/58441).

Signature: 

Date: 18/10/2026

Question 1

a) Legendre Polynomial Plots



As the order of the polynomial, q, increases, the more nonlinear/complex the $L_q(x)$ becomes:

1. At q=0, $L_q(x)$ devolves to a straight horizontal line $L_q(x)$ =1 over the domain [-1,1].

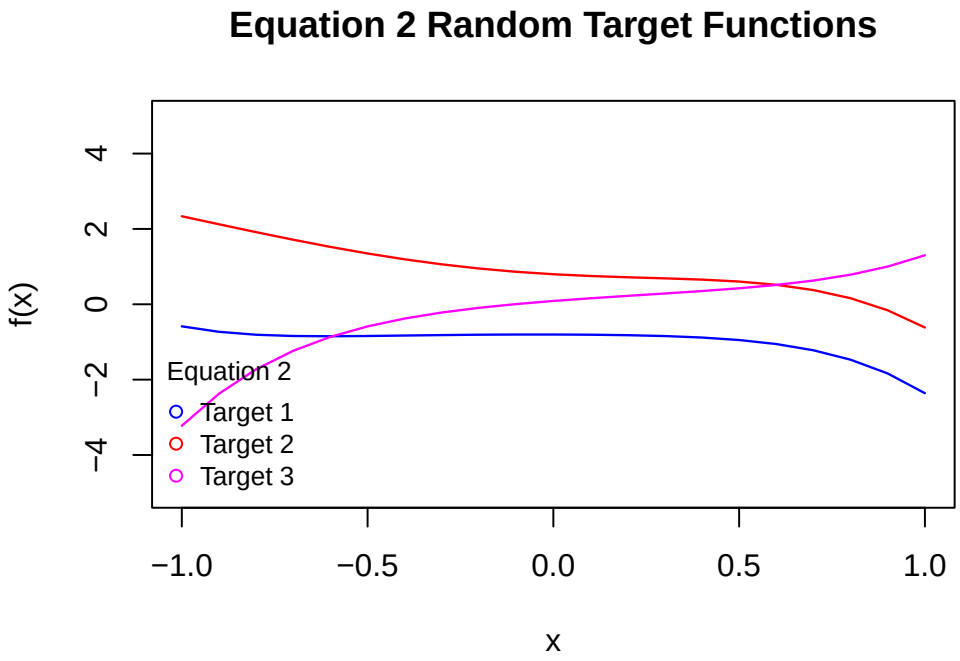
2. At q=1, $L_q(x)$ starts to have some slope attached, a straight line of the form $y = 2x$.

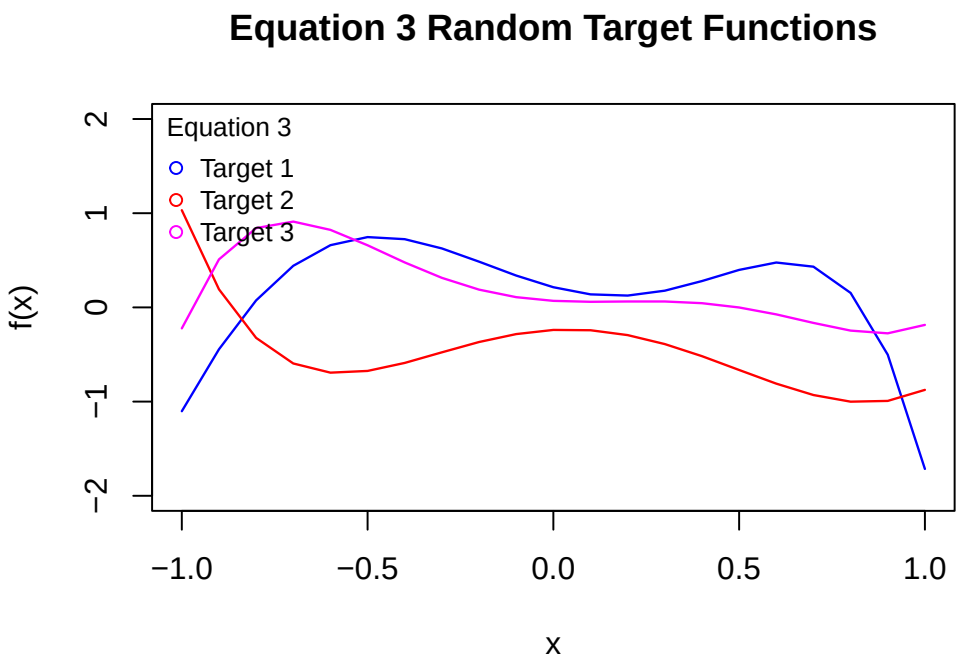
3. At q=2 the non-linear curvature/complexity of $L_q(x)$ starts to become

more visible, particularly towards the edges of the domain [-1,1]; the function approximates a parabola/quadratic function.

4. At q=3,4,5, the non-linear, polynomial curvature/complexity of $L_q(x)$ becomes more prominent with the function taking more sinusoidal shape.

b) Random Target Functions



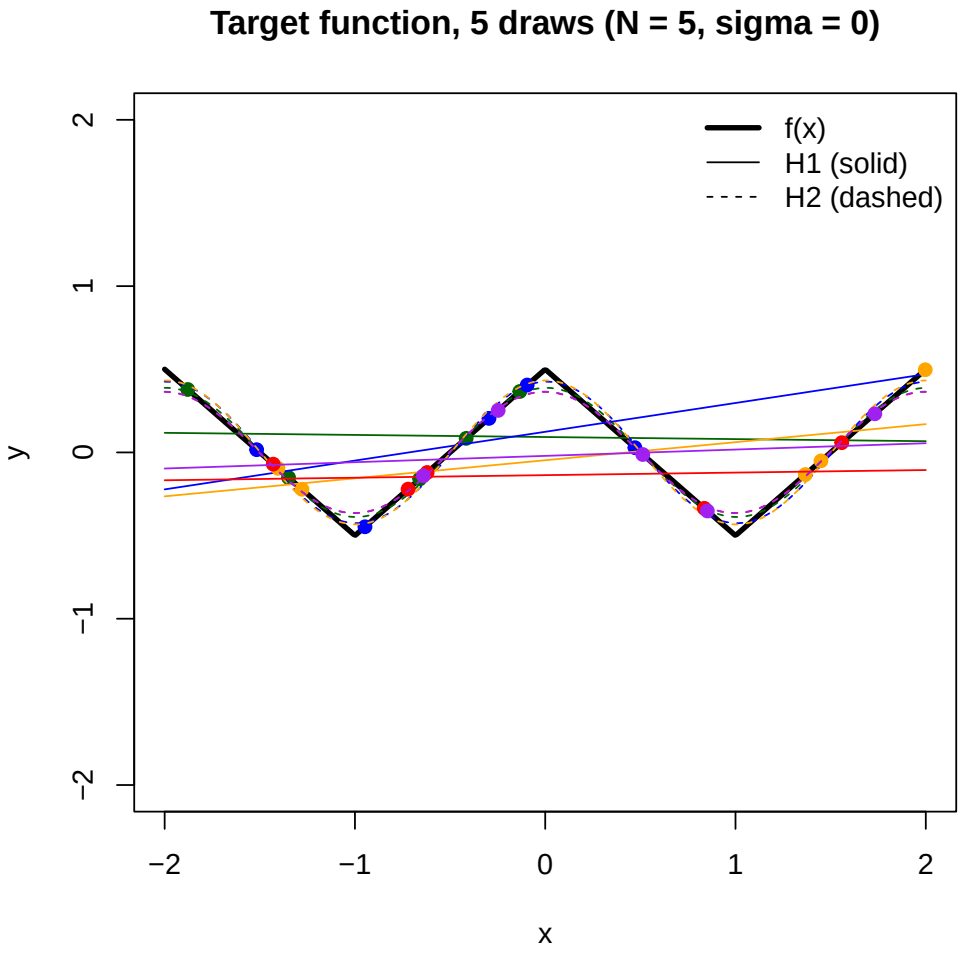


Observations:

1. $f(x) = \sum_{q=0}^{Q_f} \alpha_q x^q$: the function is smoother over the domain [-1,1], with very little curvature in between.

2. $f(x) = \sum_{q=0}^{Q_f} \beta_q L_q(x)$: there are curvatures over the same domain, with the function taking a wave-like shape courtesy of the $L_q(x)$ function.

Question 2



b) A Priori expectation

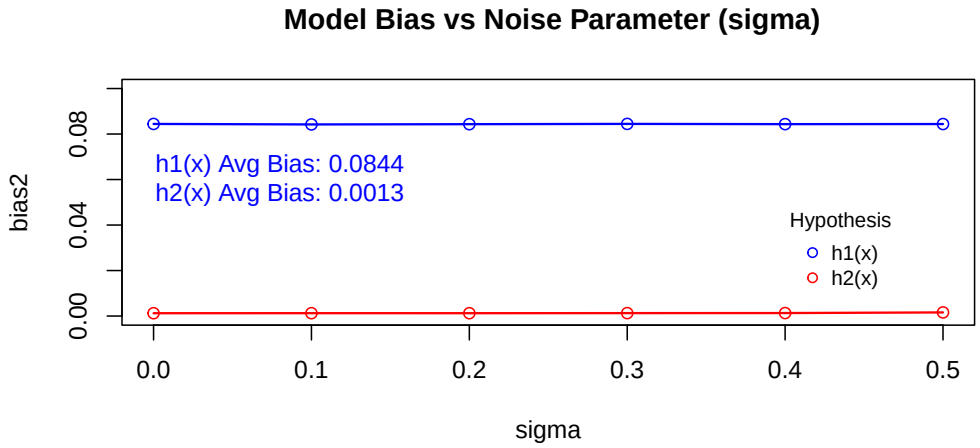
Given that the underlying data generating process is non-linear, the non-linear hypothesis would be more suited to fit the data's non-linear behavior and overall shape. The function has a non-linear zig-zag shape, with repeated peaks and troughs at between the interval which H2 seems to follow similarly across all 3 draws. H1 has the same number of parameters but can

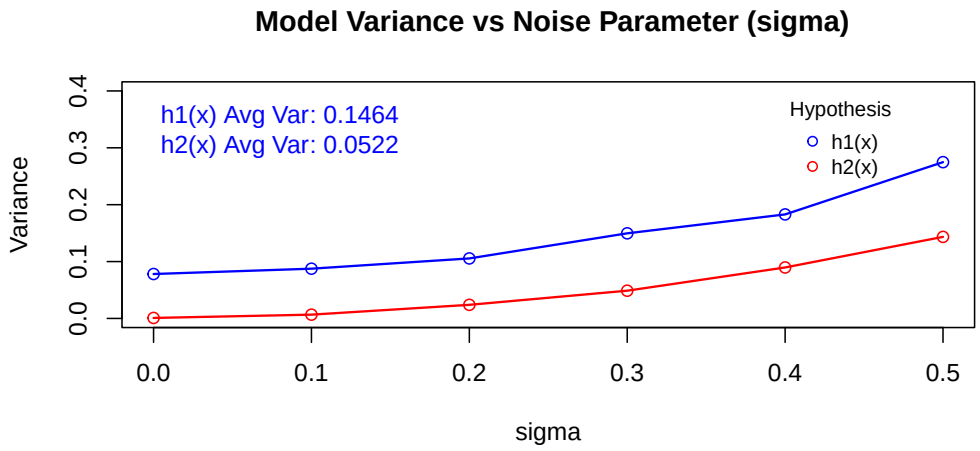
only produce a straight line, unable to capture the nonlinearity of the data generating process.

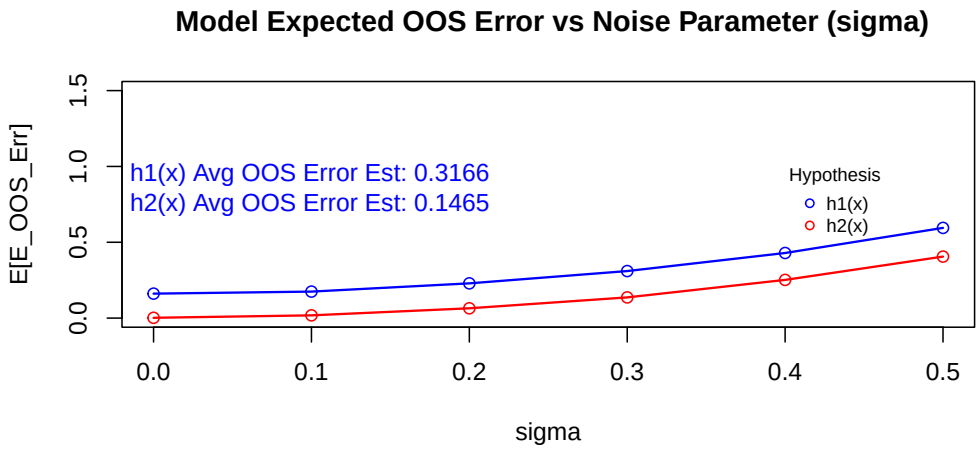
c) Bias Variance for H1 and H2

	Bias	Vari-ance	Both:Bias+Var+Noise	OOS Error	Estimate Difference
h1(x)	0.0844	0.0781	0.1626	0.1614	0.0012
h2(x)	0.0012	0.0009	0.0022	0.0020	0.0001

d) Bias Variance Dynamics







e) Interpretation

Bias-Variance Dynamics

The expected out-of-sample error as it relates to the bias and variance of a model is given by

$$E[Err_{oos}] = Bias^2 + Variance + IrreducibleNoise(\varepsilon)$$

- LHS: The model $h2(x)$ has better out-of-sample performance overall than the linear model $h1(x)$, averaged across 1000 simulations and varying noise parameters (average OOS error of 0.1465 vs 0.3166). It also has the lower error at every value of the noise parameter. The OOS error increases for both hypotheses as the noise parameter increases, with the increase growing faster at higher noise levels. This is expected, since noise adds irreducible error to the OOS observations.

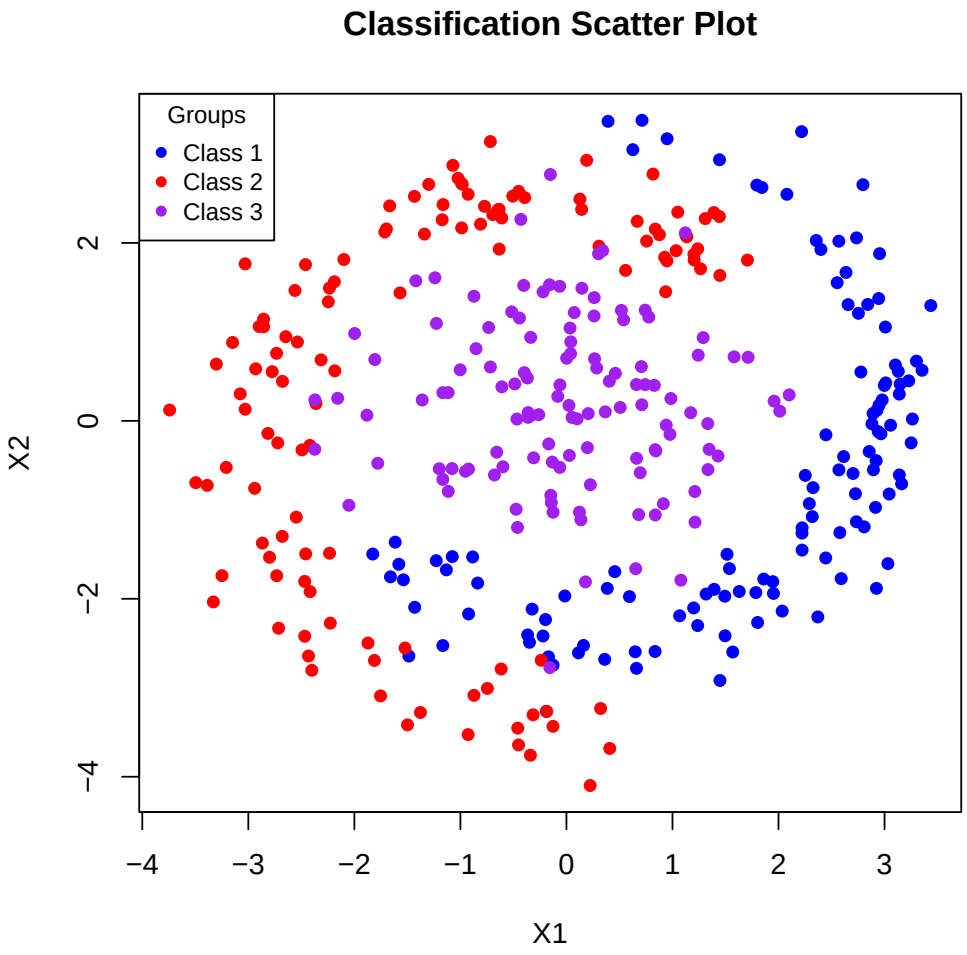
- RHS: The bias^2 is flat across the noise parameter for both functions (0.0844 for $h1(x)$ and 0.0014 for $h2(x)$), since noise oes not directly affect the level of bias in the model but is affected by the level of the model complexity (or functional form) of the hypothesis function. As expected, the bias is higher for the simpler $h1(x)$ model. The variance increases with the noise parameter for both functions, as noisier training data produces more variable fits. $h1(x)$ has the higher average variance (0.1464 vs 0.0522), offering greater stability of predictions across simulations.

  Model Choice

  The form of $h2(x)$ simply matches the target function much better than a straight line. Given the estimated average OOS error, it's better to go with the model as it is still parsimonious (2 parameters) and has greater stability and accuracy on unseen data. Also, controlling for a relatively stable bias, the variance is much more sensitive to noise, which in turn mostly affects the overall error of the model for different levels of noise.

Question 4

a) Data Set Plot



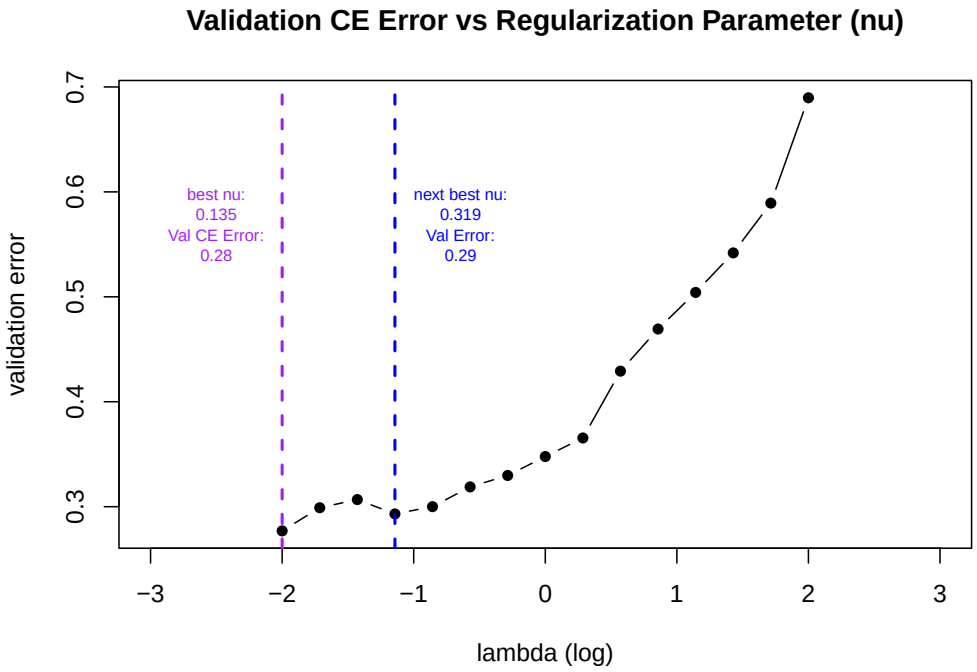
Observation: The data does not appear to be linearly separable (in the given 2D feature space) as the data does not seem to have linear boundaries between categories/classes. Instead, the different classes of responses look to be spiraled around each other. A non-linear neural network seems well suited to capture the non-linear pattern/structure embedded in the data for the given feature space.

c) Feed Forward Neural Network

Numerical Pitfalls: The computation of $log(\hat{y})$ in the objective function may cause the network to explode/ diverge given the undefined behavior of the log function at 0 (or values very close to 0), which can hamper the numerical stability of the network's computations.

d) Validation Analysis

Plot the validation error vs log (λ)



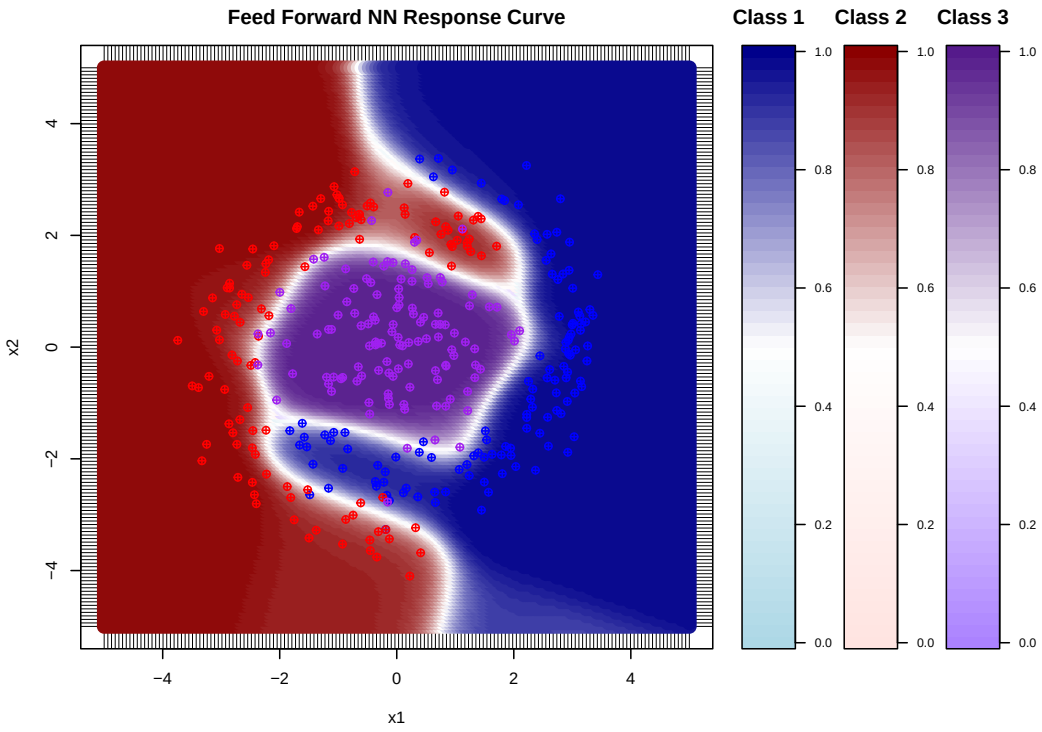
Model Parameters: number of hidden layer nodes, **m =10**

The lowest validation error is generated at $\lambda_{min} = 0.135$ with an associated validation error of 11.04, making for a less strict and more complex model (given the big budget that the sum of the model's parameters cannot exceed). The next best error to the right of this minimum is obtained at $\lambda_{1SE} = 0.319$, offering stricter control over the complexity of the network and thus more parsimonious model without too much of a deterioration on the validation

error, decreasing the likelihood of the model over fitting to just the training data. **This is the better choice of regularization parameter.**

d) Response Curve

Model Hyperparameters: **m=10, nu=0.319**



The model produces three distinct, somewhat smooth regions over the feature space, each corresponding to the probability assignments for one class:

- Class 1: Most of the Class 1 observations are correctly classified. The Class 1 region covers the right side of the feature space (roughly X1 > 0 to 1, depending on X2). The model is very confident within this region, assigning probabilities close to 1. The few misclassified observations lie near the boundary with Class 2, mainly at the top (X1 between about -1 and 0, X2 between about 2 and 3). For new observations that lie in that region near the boundary, the model will likely misclassify points as belonging to Class 2 (at worst).

- Class 2: The Class 2 region covers the left side of the feature space (roughly X1 < -1 to 0, depending on X2) and wraps above and below the Class 3 region. The model predicts Class 2 there with probabilities close to 1. Most Class 2 observations are correctly classified, with a few edge cases at the boundaries with Class1 and Class 3. These exceptions are near the white band at the top (X1 between about 0 and 1, X2 between about 2 and 3) and a few that spill into the Class 3 region.
- Class 3: The Class 3 region is a compact area in the center of the feature space (roughly X1 between -2 and 2, X2 between -1.5 and 1.5). Most Class 3 observations fall within it, with the misclassified ones mostly along its edges, in the Class 2 region on the left and the Class 1 region on the right. Probabilities within the inner region are high as well; predictions on new observations that fall within these bounds would likely behave similarly.

Appendix

Question 1 code

```
lq=function(x,q){
  coeff=2**q
  summands_store=c()
  for(k in 0:q) { # These are the ks we are summing
   ↪  over


 ↪  summands_k=(x**k)*(choose(q,k)*choose(((q+k-1)/2),q)
 ↪  )
   summands_store=c(summands_store,summands_k)
  }

  f_x=coeff*sum(summands_store)
```

```
  return(f_x)

}




# plot the above
# USE sapply to vectorize the calculation across a
 ↪  supplied vector
x_list=list(x=seq(-1,1,0.1))

x=seq(-1,1,0.1)
Qs=seq(0,5,1)

q_cols=c('blue','red','magenta','lightblue','purple','green')



par(mfrow = c(2, 3),oma = c(0, 0, 3, 0))

for(i in 1:length(Qs)){
 ↪  #c(min(sapply(x,lq,q=5)),max(sapply(x,lq,q=5)))
  q_i=Qs[i]

   ↪  plot(x,sapply(x,lq,q=q_i),col=q_cols[i],lwd=1.2,xlab='x',ylab='
   ↪  paste('Lq(x) q=',q_i),type='l')


}

mtext("Legendre Polynomials of order q", side = 3,
 ↪  outer = TRUE, cex = 1.5, font = 2)

#ya
# plots
```

```
#| label: fig-subplots
#| fig-cap: "f(x) two ways"
#| fig-subcap:
#|   - "f(x): Equation 2"
#|   - "f(x): Equation 3"
#| layout-ncol: 2


set.seed(30)

# We hace Qf coefficients


target_alpha=function(x,alpha_q,Qf){

  store_q=c()
  for(q_i in 0:Qf){

    store_q=c(store_q, (alpha_q[q_i+1]*(x**q_i)))
  }

  ret=sum(store_q)
  return(ret)
}



target_beta=function(x,beta_q,Qf){
  store_q=c()
  for(q_i in 0:Qf){
    Lq=lq(x,q_i)
    store_q=c(store_q,beta_q[q_i+1]*Lq)
  }
```

```
  ret=sum(store_q)
  return(ret)
}


# Plot the above target_alpha

plot(NULL,xlim=c(-1,1),ylim=c(-5,5),xlab='x',ylab='f(x)',main='Equati
 ↪  2 Random Target Functions')

q_cols=c('blue','red','magenta','lightblue','purple','green')

#draw aplha matrix
num_draws=3

for(i in 1:3){
  Qf=5
  alpha_q_i=runif(Qf + 1, -1, 1)

  lines(x,sapply(x,target_alpha,alpha_q=alpha_q_i
   ↪  ,Qf=5),col=q_cols[i],lwd=1.2)
}


legend("bottomleft",
       legend = c('Target 1','Target 2','Target 3'),
        ↪  # Text labels
       col = q_cols[1:3],
       pch = 1,
       cex=0.85,
       title = "Equation 2",
       bty='n')

# Plot the above target_beta
```

```
plot(NULL,xlim=c(-1,1),ylim=c(-2,2),xlab='x',ylab='f(x)',main='Equati
 ↪  3 Random Target Functions')

for(i in 1:3){
  Qf=5
  beta_q_i <- runif(Qf + 1, -1, 1)

   ↪  lines(x,sapply(x,target_beta,beta_q=beta_q_i,Qf=5),col=q_cols[i
}


legend("topleft",
       legend = c('Target 1','Target 2','Target 3'),
        ↪  # Text labels
       col = q_cols[1:3],
       pch = 1,
       cex=0.85,
       title = "Equation 3",
       bty='n')
```

Question 3 code

```
set.seed(2023)
f_x=function(x,sig=0){
  epsi=rnorm(1,0,sig)

  if(x<0){
    target=abs(x+1)-0.5
    return (target+epsi)

  }
  if(x>=0){
    target=abs(x-1)-0.5

    return(target+epsi)
```

```
  }
}

# Simulte 5 time
f_x=function(x,sig=0){
  epsi=rnorm(1,0,sig)

  if(x<0){
    target=abs(x+1)-0.5
    return (target+epsi)

  }
  if(x>=0){
    target=abs(x-1)-0.5

    return(target+epsi)

  }
}

N=5
xg=seq(-2, 2, length.out = 400)   # grid for plotting
cols=c("darkgreen", "blue", "orange", "red",
 ↪  "purple")

plot(xg, sapply(xg,f_x), type = "l", lwd = 3, ylim =
 ↪  c(-2, 2),   # widen if curves are clipped
     xlab = "x", ylab = "y", main = "Target function,
      ↪  5 draws (N = 5, sigma = 0)")

set.seed(2023)
for (i in 1:5) {
  xi=runif(N, -2, 2)
  y=sapply(xi,f_x)
  points(xi, y, pch = 19, col = cols[i])
```

```
  h1=lm(y ~ xi)
  lines(xg, predict(h1, data.frame(xi = xg)), col =
   ↪  cols[i], lty = 1)

  z1=sin(pi * xi); z2=cos(pi * xi)
  h2=lm(y ~ z1 + z2 - 1)
  lines(xg, predict(h2, data.frame(z1 = sin(pi * xg),
   ↪  z2 = cos(pi * xg))),
        col = cols[i], lty = 2)
}

legend("topright", legend = c("f(x)", "H1 (solid)",
 ↪  "H2 (dashed)"),
       lty = c(1, 1, 2), lwd = c(3, 1, 1), bty = "n")


# Funtion Arg
# f_tru: True DGP
# hyp: Hypothesis function
# sig: nr of std deviation from mean
# N: size of data
# nsim= nr of simulations/draws


fit_h = function(hyp, x, y){
  if(hyp == 'h1') fit = lm(y ~ x)
  if(hyp == 'h2') fit = lm(y ~ sin(pi * x) + cos(pi *
   ↪  x) - 1)
  return(fit)
}

oos_estimate = function(f_true, hyp, sig, N, nsim =
 ↪  1000, n_grid = 200){

  f_vec  = function(x) sapply(x, f_true)        #
 ↪  works even if f_true isn't vectorised
```

```r
 x_grid = seq(-2, 2, length.out = n_grid)     #
↪  fixed evaluation points
 f_grid = f_vec(x_grid)

 pred_store    = matrix(NA, nrow = n_grid, ncol =
↪  nsim)
 oos_err_store = numeric(nsim)

 for(i in 1:nsim){
   # training data with noise
   xi = runif(N, -2, 2)
   y  = f_vec(xi) + rnorm(N, 0, sig)
   h  = fit_h(hyp, xi, y)

   # predictions on the fixed grid (column name must
    ↪  be x)
   pred_store[, i] = predict(h, data.frame(x =
↪  x_grid))

   # fresh OOS sample with noise
   xo = runif(N, -2, 2)
   yo = f_vec(xo) + rnorm(N, 0, sig)
   oos_err_store[i] = mean((predict(h, data.frame(x
↪  = xo)) - yo)^2)
 }

 g_bar      = rowMeans(pred_store)                  #
↪  mean prediction at each grid x
 model_bias2 = mean((g_bar - f_grid)^2)
 model_var   = mean((pred_store - g_bar)^2)         #
↪  g_bar recycles down the columns
 both        = model_bias2 + model_var + sig^2      #
↪  noise variance = sig^2
 oos_err_est = mean(oos_err_store)
 estimate_diff = abs(both - oos_err_est)
```

```
  return(list(model_bias2 = model_bias2, model_var =
   ↪  model_var, both = both,
              oos_err_est = oos_err_est,
               ↪  estimate_diff = estimate_diff))
}

# Test
library(dplyr); library(tibble); library(knitr)
set.seed(321)
f_true = f_x
sig = 0
N = 5

h1_out = oos_estimate(f_true, hyp = 'h1', sig, N,
 ↪  nsim = 1000)
h2_out = oos_estimate(f_true, hyp = 'h2', sig, N,
 ↪  nsim = 1000)

comparison_table = rbind(data.frame(h1_out),
 ↪  data.frame(h2_out)) %>%
  mutate(model = c('h1(x)', 'h2(x)')) %>%
  column_to_rownames(var = "model")

kable(comparison_table, digits = 4,
      col.names = c('Bias', 'Variance',
       ↪  'Both:Bias+Var+Noise', 'OOS Error',
       ↪  'Estimate Difference'),
      align = "cc")



set.seed(321)

sigmas=seq(0,0.5,0.1)
```

```
f_true=f_x
N=5


f_x=function(x,sig=0){
  epsi=rnorm(1,0,sig)

  if(x<0){
    target=abs(x+1)-0.5
    return (target+epsi)

  }
  if(x>=0){
    target=abs(x-1)-0.5

    return(target+epsi)

  }
}

# apply function for all sigmas
h1_sigma_function_values=sapply(sigmas,oos_estimate,f_true=f_x,hyp='h

h2_sigma_function_values=sapply(sigmas,oos_estimate,f_true=f_x,hyp='h

# create bias plot
plot(NULL,xlim=c(0,0.5),ylim=c(0,0.10),xlab='sigma',ylab='bias2',main
 ↪  Model Bias vs Noise Parameter (sigma)',)
lines(sigmas,unlist(h1_sigma_function_values[1,]),col='blue',lwd=1.5)
points(sigmas,unlist(h1_sigma_function_values[1,]),col='blue',pch
 ↪  = 1)
lines(sigmas,unlist(h2_sigma_function_values[1,]),col='red',lwd=1.5)
points(sigmas,unlist(h2_sigma_function_values[1,]),col='red',pch
 ↪  = 1)
```

```
h1_oos_bias_mean=round(mean(unlist(h1_sigma_function_values[1,])),4)
h2_oos_bias_mean=round(mean(unlist(h2_sigma_function_values[1,])),4)


text3=paste('h1(x) Avg Bias:',h1_oos_bias_mean)
text4=paste('h2(x) Avg Bias:',h2_oos_bias_mean)

plot_text=paste(text3,text4,sep = "\n")

text(x =0.08, y = 0.04, labels =plot_text ,
     srt = 0, adj = c(0.5, -0.5), col = "blue",cex=1)


legend(x=0.4,y=0.05,
       legend = c("h1(x)", "h2(x)"), # Text labels
       col = c('blue','red'),
        ↪
       pch = 1,
       cex=0.85,
       title = "Hypothesis",
       bty='n')

# Plot Variance

plot(NULL,xlim=c(0,0.5),ylim=c(0,0.4),xlab='sigma',ylab='Variance',ma
 ↪  Model Variance vs Noise Parameter (sigma)',)
lines(sigmas,unlist(h1_sigma_function_values[2,]),col='blue',lwd=1.5)
points(sigmas,unlist(h1_sigma_function_values[2,]),col='blue',pch
 ↪  = 1)
lines(sigmas,unlist(h2_sigma_function_values[2,]),col='red',lwd=1.5)
points(sigmas,unlist(h2_sigma_function_values[2,]),col='red',pch
 ↪  = 1)
```

```r
h1_oos_variance_mean=round(mean(unlist(h1_sigma_function_values[2,])))
h2_oos_variance_mean=round(mean(unlist(h2_sigma_function_values[2,])))


text3=paste('h1(x) Avg Var:',h1_oos_variance_mean)
text4=paste('h2(x) Avg Var:',h2_oos_variance_mean)

plot_text=paste(text3,text4,sep = "\n")

text(x =0.08, y = 0.25, labels =plot_text ,
     srt = 0, adj = c(0.5, -0.5), col = "blue",cex=1)



legend(x=0.4,y=0.4,
       legend = c("h1(x)", "h2(x)"), # Text labels
       col = c('blue','red'),
        ↪
       pch = 1,
       cex=0.85,
       title = "Hypothesis",
       bty='n')



# Plot OOS

plot(NULL,xlim=c(0,0.5),ylim=c(0,1.5),xlab='sigma',ylab='E[E_OOS_Err]
 ↪  Model Expected OOS Error vs Noise Parameter
 ↪  (sigma)',)
lines(sigmas,unlist(h1_sigma_function_values[4,]),col='blue',lwd=1.5)
points(sigmas,unlist(h1_sigma_function_values[4,]),col='blue',pch
 ↪  = 1)
lines(sigmas,unlist(h2_sigma_function_values[4,]),col='red',lwd=1.5)
points(sigmas,unlist(h2_sigma_function_values[4,]),col='red',pch
 ↪  = 1)
```

```
legend(x=0.4,y=1.05,
       legend = c("h1(x)", "h2(x)"), # Text labels
       col = c('blue','red'),
        ↪
       pch = 1,
       cex=0.75,
       title = "Hypothesis",
       bty='n')




h1_oos_estimate_mean=round(mean(unlist(h1_sigma_function_values[4,])))
h2_oos_estimate_mean=round(mean(unlist(h2_sigma_function_values[4,])))




text3=paste('h1(x) Avg OOS Error
 ↪  Est:',h1_oos_estimate_mean)
text4=paste('h2(x) Avg OOS Error
 ↪  Est:',h2_oos_estimate_mean)

plot_text=paste(text3,text4,sep = "\n")

text(x =0.099, y = 0.55, labels =plot_text ,
     srt = 0, adj = c(0.5, -0.5), col = "blue",cex=1)
```

Question 4 code

```
dat = read.table('Collider_Data_2022.txt', h =
 ↪  TRUE,stringsAsFactors =T)
```

```
plot(NULL,
 ↪  xlim=c(min(dat['X1']),max(dat['X1'])),ylim=c(min(dat['X2']),max(d
 ↪  xlab='X1',ylab='X2',main='Classification Scatter
 ↪  Plot')

points(dat[dat$Yi1==1,]$X1,dat[dat$Yi1==1,]$X2,col='blue',pch=16)

points(dat[dat$Yi2==1,]$X1,dat[dat$Yi2==1,]$X2,col='red',pch=16)

points(dat[dat$Yi3==1,]$X1,dat[dat$Yi3==1,]$X2,col='purple',pch=16)

legend(x='topleft',
       legend = c('Class 1', 'Class 2', 'Class 3'),
       col=c('blue','red','purple'),
       pch=c(16,16,16),
       title = "Groups",
       cex=0.85,
       bty = "o")


soft_max = function(z) {  # assume z is a matrix

  z_shifted = sweep(z, 2, apply(z, 2, max), FUN =
 ↪  "-") # subtract maximum from each column to avoid
 ↪  overflow

  num = exp(z_shifted)

  # Sum exp within each col
  sums = colSums(num)

  # Normalize each column
  result = sweep(num, 2, sums, FUN = "/")

  return(result)
}
```

```
X_feat=cbind(dat$X1,dat$X2)
Y=cbind(dat$Yi1,dat$Yi2,dat$Yi3)

feedforward_nn = function(X_input, y_input, theta, m,
 ↪  nu,
                           activation_fun_2 =
                            ↪  soft_max,
                            ↪  activation_fun_1 =
                            ↪  tanh){

  y  = as.matrix(y_input)
  N  = nrow(X_input); p = ncol(X_input); d2 = ncol(y)
  X  = t(as.matrix(X_input))                          # p
 ↪  x N

  # split theta: W1, W2, b1, b2
  i1 = p * m
  i2 = i1 + m * d2
  i3 = i2 + m
  W1 = matrix(theta[1:i1],          nrow = p, ncol =
 ↪  m)
  W2 = matrix(theta[(i1 + 1):i2],   nrow = m, ncol =
 ↪  d2)
  b1 = theta[(i2 + 1):i3]
  b2 = theta[(i3 + 1):(i3 + d2)]

  b1_ones_matrix=matrix(1,nrow=1,ncol=N)
  b1_mat=b1%*%b1_ones_matrix
#
#
#   # bias matrix going  into output/layer 2
  b2_ones_matrix=matrix(1,nrow=1,ncol=N) # Repeat
 ↪  this N times
```

```
  b2_mat=b2%*%b2_ones_matrix


  # forward pass
  first_layer_comp = activation_fun_1(t(W1) %*% X+
 ↪  b1_mat)       # m x N
  second_layer_comp_no_activation = (t(W2) %*%
 ↪  first_layer_comp)+b2_mat # d2 x N
  second_layer_comp =
 ↪  activation_fun_2(second_layer_comp_no_activation)
 ↪  # d2 x N



  # cross-entropy: sum over classes and observations,
   ↪  divided by N
  CE_error    = -sum(t(y) *
 ↪  log(pmax(second_layer_comp, 1e-12))) / N
  CE_error_L2 = CE_error + (nu / N) * (sum(W1^2) +
 ↪  sum(W2^2))

  return(list(first_layer_comp = first_layer_comp,
              second_layer_comp = second_layer_comp,
              CE_error = CE_error,
              CE_error_L2 = CE_error_L2))
}
set.seed(2022)

y_input=Y
X_input=X_feat
#
m   = 10 # number nodes in hidden layer
p     = dim(X_input)[2]
q     = dim(as.matrix(Y))[2]
```

```
npars = p*m+m*q+m+q
theta_rand = runif(npars,-1,1)
N       = dim(X_feat)[1]
set     = sample(1:N,0.5*N,replace = FALSE) #50 50
 ↪  split
X_train = X_feat[set,,drop = FALSE] #
Y_train = Y[set,,drop = FALSE]
X_val   = X_feat[-set,,drop = FALSE]
Y_val   = Y[-set,,drop = FALSE]

obj_pen = function(pars)
{
  res_model =
 ↪  feedforward_nn(X_train,Y_train,pars,m,nu,activation_fun_2=soft_ma
 ↪
  return(res_model$CE_error_L2) # Reg Obj
}

n_nus = 15
nu_seq= exp(seq(-2,2,length = n_nus)) # sequence of
 ↪  nus to validate against
val_error = rep(0,n_nus)
for(i in 1:n_nus)
{
    nu = nu_seq[i] # Pick nu
    res_opt = nlm(obj_pen,theta_rand, iterlim = 1000)
 ↪  # Fit for that nu

    #Evaluate on Validation Set with optimal
     ↪  parameters and particular nu

    res_val=
 ↪  feedforward_nn(X_val,Y_val,res_opt$estimate,m,nu,activation_fun_2

    # Get reg validation Error
```

```
  val_error[i] = res_val$CE_error

  #print(paste0('Validation run:',i,' nu = ', nu))
}


#| fig-dpi: 300


min_val_error=min(val_error)



best_nu_index=which(val_error==min(val_error))
best_nu=nu_seq[best_nu_index];


# Compare the error of the nu the right of the best

next_val_error=val_error[(best_nu_index+1):length(val_error)]

next_seq=nu_seq[(best_nu_index)+1:length(nu_seq)]

next_best_nu=next_seq[which.min(next_val_error)] ;

next_min_val_error=min(next_val_error)




plot(val_error~log(nu_seq), type = 'b',pch =
 ↪  16,xlim=c(min(log(nu_seq))-1,max(log(nu_seq))+1),
 ↪  main='Validation CE Error vs Regularization
 ↪  Parameter (nu)', xlab='lambda
 ↪  (log)',ylab='validation error')
```

```
abline(v =log(best_nu), col = "purple", lty =
 ↪  "dashed", lwd = 2)

abline(v =log(next_best_nu), col = "blue", lty =
 ↪  "dashed", lwd = 2)

text(x = log(best_nu)-0.5, y = 0.5, labels =
 ↪  paste("best nu:",round(best_nu,3), 'Val CE
 ↪  Error:',round(min_val_error,2),sep = "\n"),
     srt = 0, adj = c(0.5, -0.5), col =
      ↪  "purple",cex=0.68)


text(x = log(next_best_nu)+0.5, y = 0.5, labels =
 ↪  paste("next best nu:",round(next_best_nu,3), 'Val
 ↪  Error:',round(next_min_val_error,2),sep = "\n"),
     srt = 0, adj = c(0.5, -0.5), col =
      ↪  "blue",cex=0.68)
#
#


library(colorspace)

# Fit probability color mapping for each class onto
 ↪  the feature space



# Helper functions

# Assign color pallet based on indexing
  ass_pallet=function(ind_val,col_steps=50){
    if(ind_val==1){
```

```
      # Class 1, high to low prob color mapping
      pallet=colorRampPalette(c("lightblue",
 ↪  'white',"darkblue"))

    }

    if(ind_val==2){

      # Class 2
      pallet=colorRampPalette(c("mistyrose",'white',
 ↪  "darkred"))

    }
   if(ind_val==3){

     # Class 3

 ↪  pallet=colorRampPalette(c("mediumpurple1",'white',
 ↪  "purple4"))

   }

    return(pallet(col_steps))
  }


# Assign a color to prediction based on its class and
 ↪  strength of probability value

col_pred=function(x,col_steps=50){
  # x=prediction matrix from ff nn

  max_ind=apply(x,2,which.max) #get index of max
 ↪  probability
  max_prob=apply(x,2,max) # get value of max
 ↪  probability
```

```
  col_range=seq(0,1,length.out = col_steps)
  col_pallet=sapply(max_ind,ass_pallet) #prob colors
 ↪  for each class, matrix
  color_indices= findInterval(max_prob, col_range) #
 ↪  for each obs, find its indx in a range of 0 to 1

  col_pallet_N=ncol(col_pallet)
  selected_cols= col_pallet[cbind(color_indices,
 ↪  seq_len(col_pallet_N))] # get appropriate index
 ↪  in the pallette assignment matrix

  return(selected_cols)
}

pal1 = ass_pallet(1)
pal2 = ass_pallet(2)
pal3 = ass_pallet(3)

obj_pen = function(pars)
{
  res_model
 ↪  =feedforward_nn(X_input,y_input,pars,m,next_best_nu,activation_fu
  return(res_model$CE_error_L2) # Reg Obj
}

# Optimize weights using next best nu
res_opt_best_nu = nlm(obj_pen,theta_rand, iterlim =
 ↪  1000)



M   = 160
x1  = seq(-5,+5,length = M)
x2  = seq(-5,+5,length = M)
xx1 = rep(x1,M)
xx2 = rep(x2,each = M)
```

```
layout(matrix(1:4, nrow = 1),
       widths = c(5, 0.7, 0.7, 0.7))

par(mar = c(4, 4, 3, 1))

plot(NULL,
 ↪  xlim=c(-5,5),ylim=c(-5,5),xlab='x1',ylab='x2',
 ↪  main='Feed Forward NN Response Curve')

abline(h= x2,v = x1, lwd = 0.3,col='black')




XX = cbind(xx1,xx2)
YY1 = rep(1,M^2)
YY2=rep(1,M^2)
YY3=rep(1,M^2)
YY=cbind(YY1,YY2,YY3)


# Again use next best nu
res_fitted=feedforward_nn(XX,YY,res_opt_best_nu$estimate,m,next_best_
 ↪

points(xx2~xx1,pch = 16,
 ↪  col=col_pred(res_fitted$second_layer_comp) ,cex =
 ↪  2)

points(dat[dat$Yi1==1,]$X1,dat[dat$Yi1==1,]$X2,col='blue',pch=10)

points(dat[dat$Yi2==1,]$X1,dat[dat$Yi2==1,]$X2,col='red',pch=10)

points(dat[dat$Yi3==1,]$X1,dat[dat$Yi3==1,]$X2,col='purple',pch=10)
```

```r
add_colour_bar = function(pal, class_name) {

  par(mar = c(4, 0.5, 3, 2))

  probs <- seq(0, 1, length.out = length(pal))

  # Draw continuous-looking gradient
  z = matrix(rep(probs, each = 2), nrow = 2)

  image(x = c(0, 1),
        y = probs,
        z = z,
        col = pal,
        zlim = c(0, 1),
        axes = FALSE,
        xlab = "",
        ylab = "",
        main = class_name)

  axis(4, at = seq(0, 1, by = 0.2),
       las = 1, cex.axis = 0.75)

  box()
}

#colour mapping
add_colour_bar(pal1, "Class 1")
add_colour_bar(pal2, "Class 2")
add_colour_bar(pal3, "Class 3")
```